講義:用強化學習教機械手握拳與捏合 —— 演算法與超參數的系統性比較

> **這份講義怎麼讀**
> 這是一份「上課講義」,目標是讓沒學過強化學習的人也能跟上,同時把這次實驗的每一個決定都交代清楚。
> 每一節開頭會先說「這一節要回答什麼問題」,結尾有「本節重點」。
> 看到 `檔名.py` 或 `函式名稱()`,代表可以打開專案裡的程式碼對照著看。
> 同一批數據的正式學術寫法見 `論文_ORCA機械手強化學習演算法與超參數比較.md`;
> 每個實驗的原始紀錄(含所有數字與失敗的嘗試)見 `實驗紀錄.md` (E01–E23)。

目錄

1. 背景與動機
2. 理論基礎:強化學習與三種演算法
3. 實驗設計:任務怎麼定義、要比什麼、怎麼比
4. 實驗結果
5. 結果討論:為什麼會這樣?
6. 結論與後續建議
7. 課後思考題
8. 附錄:常用指令與程式地圖

1. 背景與動機

> **本節要回答**:我們在做什麼?為什麼要「比較」演算法,而不是隨便挑一個來用?

1.1 我們要解決的問題

我們有一隻模擬的機械手——**ORCA Hand**(右手),它有 **17 個可以動的關節**:手腕 1 個、拇指 4 個、其他四根手指各 3 個(左右張開、根部彎曲、中段彎曲)。我們想讓它學會兩個動作:

- **握拳**:四根手指彎下來碰到掌心,拇指收進來壓在手指上。
- **捏合**:拇指指尖碰到食指指尖(像在比「OK」)。

聽起來很簡單,但想像一下要你**寫程式**告訴 17 個關節每一瞬間該轉幾度——而且手指之間會互相碰撞、擋住——這其實很難手動設計。

1.2 強化學習的想法

強化學習(Reinforcement Learning, RL) 換了一個角度:我們不告訴機械手「怎麼做」,只告訴它「做得好不好」(給分數),讓它自己反覆嘗試、慢慢找出做法。

> **比喻**:訓練小狗握手。你不會去扳小狗的每一塊肌肉,而是牠做對了就給零食。久了牠自己就知道怎麼做。

1.3 為什麼要比較?

RL 有很多種演算法(就像訓練小狗有很多種方法),每種都有各自的超參數(像是「每次給多少零食」「多久訓練一次」)。問題是:

1. **不同的演算法在不同任務上表現差很多**,別人論文裡最好的,不一定在我們的機械手上最好。
2. **超參數調不好可能完全學不起來**,但調參數很花時間。

所以這次專案的目標是:**有系統地比較** PPO、SAC、TD3 三種演算法,再針對表現最好的那個調整超參數,找出最適合這兩個任務的組合。

1.4 這次實驗的「意外收穫」

原本以為這是個單純的「比較」實驗,做下去才發現,**最大的坑不在演算法,而在任務本身的定義**:最初寫的握拳和捏合的成功條件,在物理上**根本不可能達成**。這段「試錯 → 發現問題 → 修正 → 驗證」的過程,是這份講義第 3 節的重點,也是最值得學的部分。

> **本節重點**
>
> - 目標:讓 17 個關節的模擬機械手學會握拳和捏合。
> - 方法:強化學習——只給分數,讓它自己學。
> - 比較的對象:3 種演算法(PPO、SAC、TD3)+ 最佳演算法的 3 種超參數。
> - 意外發現:任務定義本身就有問題,要先修好才能比較。

2. 理論基礎:強化學習與三種演算法

> **本節要回答**:強化學習到底怎麼運作?PPO、SAC、TD3 有什麼不同?

2.1 強化學習的五個基本元素

元素	意思	在我們的機械手上
Agent(代理人)	做決定、要學習的程式	控制機械手的「大腦」(一個神經網路)
環境(Environment)	Agent 互動的對象	MuJoCo 模擬出來的機械手
觀測(Observation)	Agent 每一步看到什麼	34 個數字:17 個關節角度 + 17 個關節速度
動作(Action)	Agent 每一步做什麼	17 個數字:每個關節的目標角度
獎勵(Reward)	每一步的分數	例如「指尖離目標多遠」的負數,越接近 0 越好

運作方式是一個循環:**看觀測 → 決定動作 → 環境變化 → 拿到獎勵 → 再看觀測……** 每一個循環叫做一**步(step)**,我們的設定裡一步代表模擬時間 0.01 秒。從開始到成功(或時間到,最多 300 步 = 3 秒)叫做一個**回合(episode)**。

Agent 的目標不是「這一步分數最高」,而是**「整個回合加起來的分數最高」**。這很重要:有時候要先做一個看起來沒加分的動作(例如先把四指彎下去),後面才能拿到大分(拇指收進來、成功)。

> **程式對照**: `orca_sim` 套件的 `OrcaHandRight` 就是環境,它有 `reset()` (回到初始姿勢)和 `step(action)` (執行一步)。我們的任務類別 `OrcaFistTaskV2`(`fist_task_v2.py`)和 `OrcaPinchTaskV2`(`pinch_task_v2.py`)繼承它,覆寫 `_get_reward()` (怎麼給分)和 `_get_terminated()` (什麼時候算成功)。

2.2 策略、Actor 和 Critic

- **策略(Policy)**:看到觀測後決定動作的規則。在深度強化學習裡,策略是一個神經網路,又叫 **Actor(演員)**。
- **Critic(評論家)**:另一個神經網路,負責評估「在這個狀態做這個動作,將來大概能拿多少分」。

> **比喻**:Actor 是學開車的學生,Critic 是坐在旁邊的教練。學生負責操作,教練說「剛剛那個轉彎不錯」或「這樣會撞到」。學生根據教練的評語調整自己。

2.3 最重要的一個區分:On-policy vs Off-policy

這是理解這次實驗結果的關鍵。

	On-policy(PPO)	Off-policy(SAC、TD3)
用什麼資料學習	只用**目前這個策略**剛收集的資料	用**過去所有時候**收集的資料
資料用完後	丟掉	存在 **Replay buffer(經驗回放緩衝區)**,反覆抽出來學
比喻	每次考完試,只拿**這次**的考卷檢討,檢討完就丟	有一本**錯題本**,每天從裡面抽題目複習
需要的互動步數	多(資料只用一次)	少(資料重複利用)
每一步的計算量	小	大(每收一步就要從錯題本抽一批來學)

記住這張表——第 4、5 節的「樣本效率 vs 牆鐘時間」的結論,就是從這裡來的。

2.4 PPO(Proximal Policy Optimization)

核心想法:每次只改一點點。

PPO 收集一批經驗之後,會調整策略讓「好的動作」更容易被選到。但它有一個限制:**新策略不能跟舊策略差太多**。具體來說,它計算「新策略選這個動作的機率 ÷ 舊策略選這個動作的機率」,如果這個比值超出 0.8 到 1.2 的範圍,就不再繼續往那個方向推。

> **為什麼要限制?** 想像你在學投籃,今天投進了一顆,如果你因此把整個姿勢大改,明天可能一顆都投不進。PPO 的「小步調整」讓學習比較穩定。

PPO 的策略是隨機的:它輸出的是「動作的平均值」加上「一點隨機性」(高斯分布)。訓練時用隨機的版本(才會嘗試新動作),評估時通常只取平均值(稱為 deterministic,確定性)。這一點在第 5.4 節會變得很重要。

SB3 的預設設定:每個環境收集 2048 步就更新一次、網路是 64×64(兩層、每層 64 個神經元)。

2.5 SAC(Soft Actor-Critic)

核心想法:一邊追求高分,一邊刻意保持隨機性。

SAC 是 off-policy 演算法(有錯題本)。它的特別之處是目標函數裡多了一項 **「熵(entropy)」**——也就是策略的隨機程度。SAC 想要的是「分數高,而且不要太早變得死板」。

> **比喻**:一個餐廳評論家,除了找好吃的餐廳,也刻意每週嘗試一家沒去過的。這樣比較不會錯過更好的餐廳。

隨機性要多少由一個係數 α 決定,SB3 預設會**自動調整**(`ent_coef="auto"`)。SAC 也用兩個 Critic、取比較保守(比較小)的那個估計,避免太樂觀。

SB3 的預設設定:網路 256×256、學習率 3×10^{-4}、每次從錯題本抽 256 筆(batch size)。

2.6 TD3(Twin Delayed DDPG)

核心想法:確定性策略 + 三個讓訓練更穩的技巧。

TD3 也是 off-policy,但它的策略是**確定性的**:同樣的觀測永遠輸出同樣的動作。它的三個技巧:

1. **Twin(雙胞胎 Critic)**:兩個 Critic 都估一次,取比較小的。避免「高估某個動作有多好」。
2. **Delayed(延遲更新)**:Critic 更新 2 次,Actor 才更新 1 次。先讓教練的評語準一點,學生再跟著調整。
3. **目標平滑**:Critic 在估計時,對動作加一點小雜訊,讓它對動作的小變化不要太敏感。

> **一個必須說清楚的細節**(實驗紀錄 E23):原始 TD3 論文在**訓練時**會對動作加隨機雜訊來探索。但我們使用的 SB3 函式庫預設 `action_noise=None` (不加),我們也沿用預設。所以我們的 TD3 只有在最開始的 100 步用完全隨機的動作,之後**沒有額外的探索**。這是在撰寫講義時查原始碼才發現的,先前文件寫錯的地方已經更正。

SB3 的預設設定:網路 400×300、學習率 1×10^{-3}、batch size 256。

2.7 三種演算法一覽

	PPO	SAC	TD3
類型	On-policy	Off-policy	Off-policy
策略	隨機(高斯)	隨機(刻意保持熵)	確定性
有錯題本嗎	沒有	有	有
預設網路	64×64	256×256	400×300
預設學習率	3×10^{-4}	3×10^{-4}	1×10^{-3}
預期優點	穩定、每步計算快	樣本效率高、探索充分	樣本效率高
預期缺點	需要很多步	每步計算慢	每步計算慢

2.8 超參數是什麼?

超參數是演算法裡需要「人事先設定」的數值,不會自己學。這次比較了三個:

超參數	白話解釋	調太大	調太小
學習率	每次更新時參數改多少	學得快但容易「矯枉過正」、不穩定	很穩但學很慢
網路大小	神經網路有多少神經元	能力強但訓練慢	訓練快但可能學不起來複雜的東西
Batch size	每次更新時從錯題本抽幾題	每次更新比較準但比較慢	比較快但比較「吵」(不準)

> **本節重點**
>
> - RL = 觀測 → 動作 → 獎勵 的循環,目標是整個回合的總分最高。
> - **On-policy(PPO)資料只用一次;Off-policy(SAC、TD3)有錯題本重複用。** 這決定了「要多少步」和「每步多慢」。
> - PPO:小步調整、隨機策略。SAC:保持隨機性的 off-policy。TD3:確定性策略 + 雙 Critic + 延遲更新。
> - 我們的 TD3 沒有加探索雜訊(SB3 預設)。

3. 實驗設計:任務怎麼定義、要比什麼、怎麼比

> **本節要回答**:握拳和捏合的「分數」與「成功」要怎麼定義?為什麼一開始的定義不能用?實驗要怎麼設計才公平?

這一節是整個專案最多「試錯」的地方。我們依照實際發生的順序來講。

3.1 第一個坑:原本的任務定義根本做不到(E04)

專案一開始的定義是:

- **握拳 v1**:手動寫下一組「握拳時 17 個關節應該是幾度」的目標值,分數 = 負的「目前角度跟目標的距離」,距離 < 0.05 就算成功。
- **捏合 v1**:分數 = 負的「拇指末節跟食指末節的距離」,< 15 mm 就算成功。

在開始比較演算法前,我們先問了一個問題:**「這個成功條件,在物理上真的做得到嗎?」**

怎麼檢查?用一個叫 **CEM(交叉熵法)** 的最佳化方法:不用 RL,直接暴力搜尋「什麼樣的馬達指令能讓距離最小」。

> **CEM 怎麼運作**:隨機產生 48 組指令 → 每組都實際模擬看看結果 → 留下最好的 8 組 → 根據這 8 組調整下一輪的抽樣範圍 → 重複 25–30 輪。它找到的就是「物理上大概能做到的最好結果」。
> **程式對照**: `demo_reward_v2.py` 的 `cem()`；`settle()` 是「重設環境後固定送同一組指令 80 步,讓手停下來」。

結果:

任務	成功門檻	直接命令到目標	CEM 找到的最佳	之前訓練的 PPO 模型
握拳	距離 < 0.05	0.736	0.660	0.706
捏合	< 15 mm	—	17.6 mm	24.3 mm

兩個都做不到,怎麼樣都差一截。為什麼?

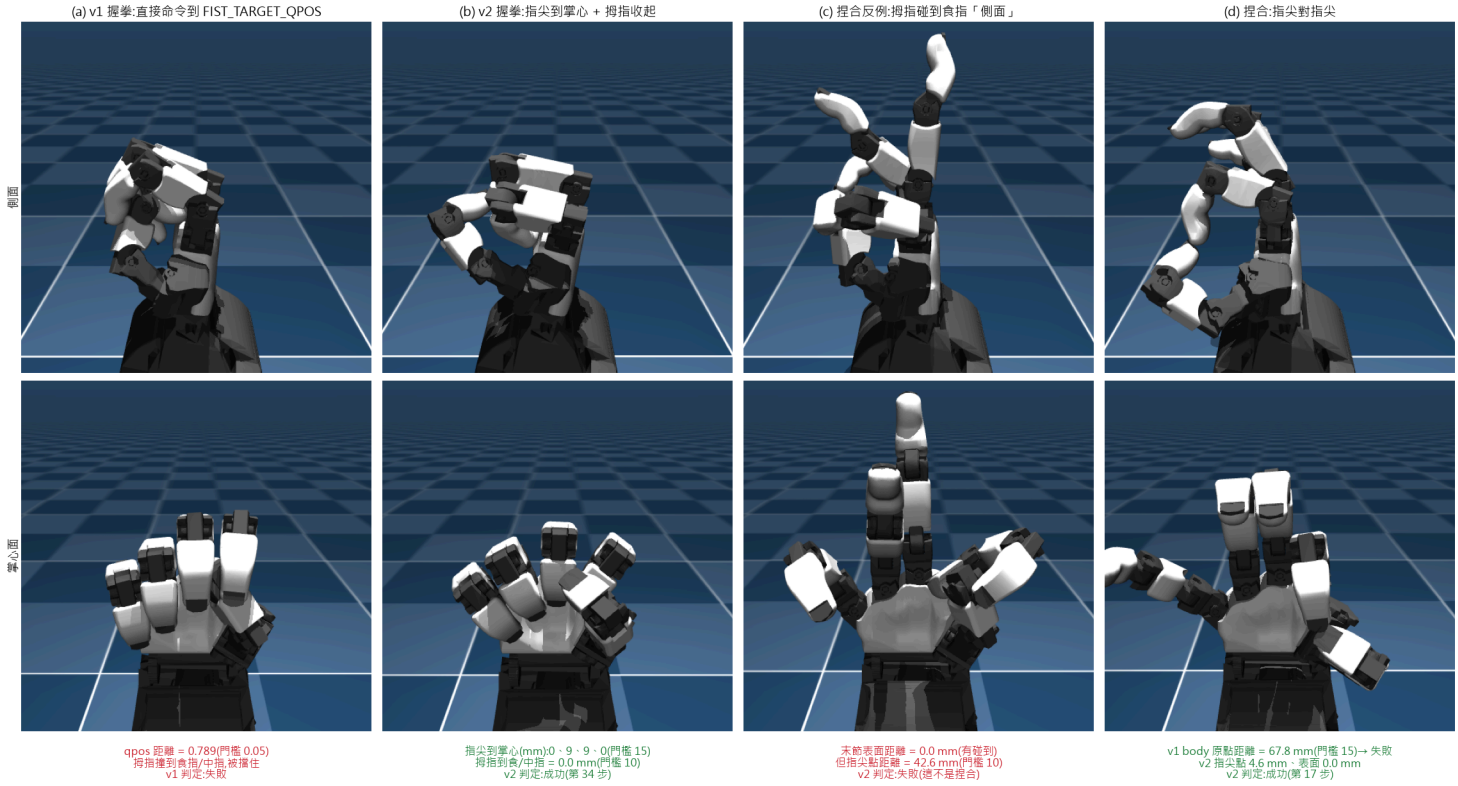
- **握拳**:我們看了模擬裡的「接觸點列表」,發現拇指壓在食指、中指上。然後做了一個關鍵的對照實驗:**把碰撞關掉**,距離馬上降到 0.019(成功)。結論:手動寫的目標姿勢會讓拇指**穿過**其他手指——在沒有碰撞的世界可以,在真實物理裡不行。
- **捏合**:CEM 找到的最佳姿勢裡,兩指的表面**已經碰在一起了**,但距離還是 17.6 mm。原因是程式量的是 MuJoCo 的 `xpos` ——**body 的座標原點,它在關節附近,不是指尖**。

> **這個坑的教訓**:如果沒做這個檢查,直接拿 v1 去比較演算法,所有演算法的成功率都會是 0%,花了幾天的運算時間卻什麼都比不出來。**在訓練前花幾分鐘檢查成功條件是否可行,非常值得。**
> **程式對照**:這個檢查的完整重現在 `analysis/verify_v1_infeasible.py` 。

3.2 捏合的修正:從「表面碰到」到「指尖對指尖」(E05、E06)

第一次修正:改量兩指**表面**之間的最近距離(用 MuJoCo 的 `mj_geomDistance()`),< 2 mm 就算碰到。

我們用 CEM 找了一個「成功」的姿勢,然後——**把它畫出來看**。結果發現:



看圖 (c):拇指碰到的是食指的**側面**,不是指尖!數字上「表面距離 = 0」完全通過,但這根本不是捏合。

第二次修正(最終版):從碰撞模型的頂點算出每根手指的「**指尖點**」(手指最前端 4 mm 範圍內所有頂點的中心),成功條件改成:

1. 兩個指尖點距離 < 10 mm,**而且**
2. 兩指表面確實接觸(< 2 mm),
3. 連續維持 5 步(避免只是剛好擦過)。

CEM 驗證:指尖對指尖最近能到 4.6 mm,條件做得到;而剛才那個碰到側面的姿勢,指尖點距離是 42.6 mm,會被正確判定為失敗(圖 (d) 是真正的捏合)。

> **這個坑的教訓:數字通過不代表行為正確。** 一定要把結果畫出來看。
> **程式對照**: `task_utils.py` 的 `fingertip_local_point()` (算指尖點)、 `surface_distance()` (算表面距離); `pinch_task_v2.py` 的 `fingertip_distance()` 、 `contact_distance()` 、 `_is_pinch()` 。

捏合最終的分數(每一步):

$$\text{reward} = -\text{指尖點距離(公分)} - 0.05 \times \text{動作變化量}^2 + 5 \times [\text{成功}]$$

- 第一項:越靠近越好。
- 第二項:懲罰動作突然大幅改變,讓動作平滑。
- 第三項:成功時給一個大獎勵。

3.3 握拳的修正:不要人來指定,讓資料告訴我們(E05、E07)

握拳的問題是「人指定的目標姿勢不合法」。所以我們換個思路,分兩步:

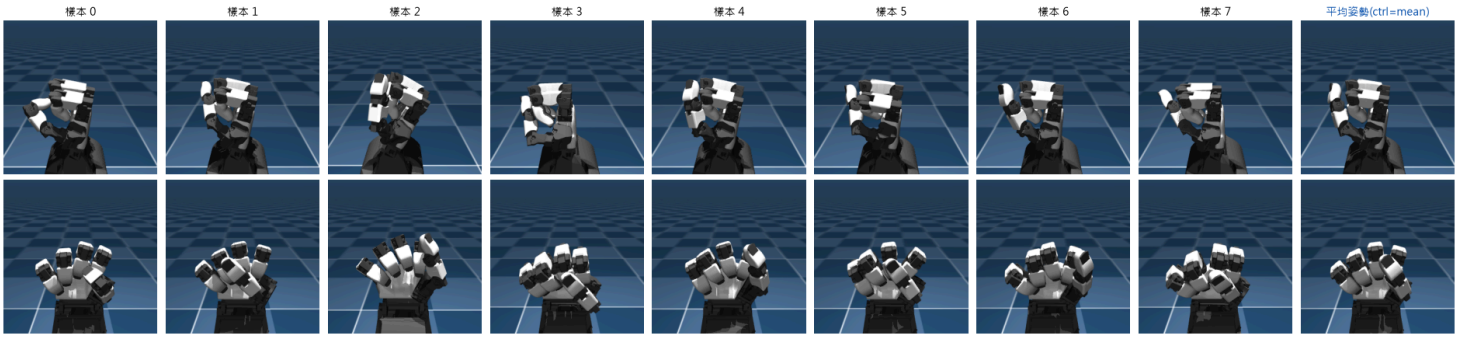
第一步:用「物理結果」定義成功。 不再看關節角度,而是看:

- 四根手指的指尖到掌心的距離都 < 15 mm(手指有握下去),
- 拇指指尖到食指或中指 < 10 mm(拇指有收進來),
- 連續維持 5 步。

為什麼是 15 mm 不是 10 mm?我們量過:四指彎到底時,中指和無名指**物理上最近只能到 9 mm 左右**(手指的形狀讓它們碰不到掌心)。原本設 12 mm,CEM 找到的成功姿勢中指是 11.3 mm,餘裕太小,所以放寬到 15 mm。**門檻要根據實測的物理極限來設**,不能憑感覺。

第二步:從合法的握拳中,統計出「標準握拳」長什麼樣子。

我們用 CEM 找了 8 組**真的成功**的握拳(第一次從完全隨機的起點找,5 次只成功 1 次,那批結果直接捨棄;改從「四指彎到底」附近出發,8 次全部成功):



統計這 8 組的關節角度,發現一件很有意思的事:

關節	8 組之間的差異(標準差)	意思
四指彎曲	幾乎沒有(≤ 0.05 rad)	**握拳 = 四指彎到底**,只有一種做法
四指左右張開	中等(0.18–0.24 rad)	有一點彈性
拇指	很大(0.22–0.63 rad)	**拇指怎麼放都可以**

這正好說明了 v1 為什麼失敗:**它硬把拇指指定在一個位置**,而那個位置剛好會撞到手指。

於是我們設計了「**加權姿勢誤差**」:每個關節的誤差,除以它的「容許範圍」(上面的標準差,最小 0.1 rad):

$$E_{\text{pose}} = \sqrt{\frac{1}{17}\sum_{\text{每個關節}}\left(\frac{\text{目前角度} - \text{標準角度}}{\text{容許範圍}}\right)^2}$$

意思是「平均差了幾個容許範圍」。四指彎曲差一點點就扣很多分,拇指差很多也只扣一點點。

> **程式對照**: `derive_fist_standard.py` 的 `collect_samples()` (收集合法樣本); `fist_task_v2.py` 的常數 `FIST_STANDARD_QPOS` (標準姿勢)、 `FIST_JOINT_STD` (容許範圍)、方法 `pose_error()` 。

3.4 第二個坑:靜態分析說 A 最好,實際訓練卻是 A 最差(E08–E11)

加權姿勢誤差的方向是對的,但我們發現它對「最後一點點彎曲」太寬容:姿勢誤差 < 1(看起來很接近了)的姿勢中,只有 20% 真的成功,多半是四指沒彎到碰到掌心。

於是我們想在分數裡多加「距離項」,比較三個版本:

版本	分數 =
pose	−姿勢誤差
pose_tip	−姿勢誤差 − 四指尖到掌心的平均距離
pose_tip_thumb	−姿勢誤差 − 四指尖距離 − 拇指距離

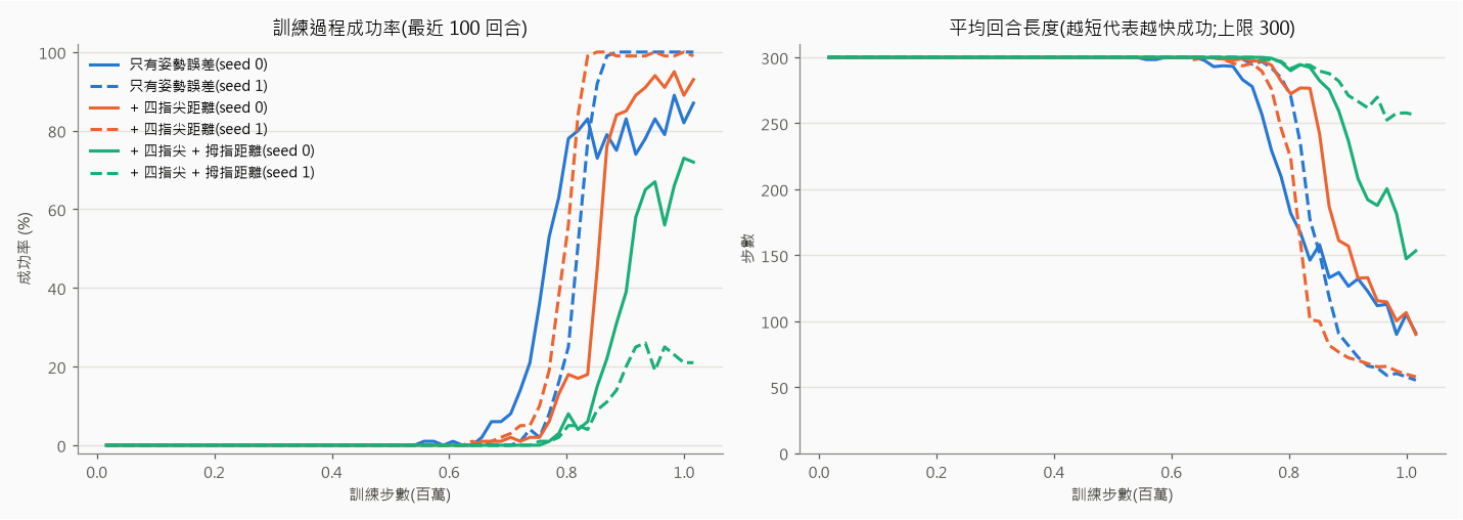
靜態分析:隨機產生 400 個姿勢,看哪個版本的分數最能區分「成功」和「失敗」的姿勢(用 AUC 衡量:隨機挑一個成功、一個失敗的姿勢,成功的那個分數比較高的機率)。

版本	AUC	分數最高的 25 個姿勢中成功的比例
pose	0.894	32%
pose_tip	0.918	40%
pose_tip_thumb	**0.956**	**48%**

看起來加拇指項最好,對吧?**但實際用 PPO 訓練 100 萬步之後**:

版本	訓練結束時的成功率(兩個 seed)
pose	87% / 100%

版本	訓練結束時的成功率(兩個 seed)
pose_tip	93% / 99%
pose_tip_thumb	**72% / 21%**(最差!)



為什麼會這樣? 我們的推測:拇指項鼓勵拇指「越早壓到手指上越好」,但如果拇指**先**壓上去,就會擋住四指彎下來。正確的順序是「先彎四指、再收拇指」。靜態分析只看「這個姿勢的分數排序對不對」,完全看不到「學習過程中的先後順序」。

> **這個坑的教訓**:**獎勵設計要用實際訓練來驗證**,靜態分析只能當參考。
> 另外,三個版本在 **30 萬步時全部是 0%**,到 100 萬步才全部學會。如果只看 30 萬步,會誤以為設計全部失敗。**下結論之前,訓練預算要給夠。**

最終握拳的分數採用 **pose_tip**:

$$\text{reward} = -E_{\text{pose}} - \text{四指尖到掌心平均距離(公分)} - 0.05 \times \text{動作變化量}^2 + 5 \times [\text{成功}]$$

3.5 讓評估有意義:初始姿勢的隨機變化(E14)

原本的環境每回合開始時,手的姿勢**一模一樣**。評估時策略又是確定性的(同樣觀測 → 同樣動作),所以「評估 20 回合」等於**同一部影片重播 20 次**——成功率只會是 0% 或 100%。

> **比喻**:考試出 20 題,結果 20 題都是同一題。

所以每回合開始時,讓每個關節隨機偏移一點點(±0.05 rad,約 ±3 度)。這樣 20 回合就是 20 個稍微不同的起點。驗證後發現學習速度幾乎不受影響。

> **程式對照**: `task_utils.py` 的 `apply_reset_noise()` ;用同一個 seed 會得到同一個起始姿勢,所以結果可重現。

3.6 CPU 還是 GPU?實測說了算(E02、E12)

一般印象是「深度學習用 GPU 比較快」。我們實際量了:

演算法	CPU	GPU	結果
PPO	~2,870 步/秒	~975 步/秒	**GPU 慢 2.9 倍**
SAC	178	105	GPU 慢 1.7 倍
TD3	218	211	差不多

為什麼 GPU 反而慢?

1. **物理模擬(MuJoCo)只在 CPU 上跑**,GPU 幫不上忙。
2. **網路太小**(64×64 這種),GPU 每次啟動運算的固定開銷,比計算本身還久。就像叫一台卡車去載一顆蘋果。
3. **每一步都要把資料在 CPU 和 GPU 之間搬來搬去**。

所以全部實驗用 CPU。

3.7 實驗設計:兩個階段

階段	比什麼	怎麼比	組數
A	3 種演算法	全部用預設超參數,兩個任務 × 3 個 seed	18
B	最佳演算法的 3 種超參數	一次只改一個,每個 2 種值 × 3 個 seed × 2 個任務	36
正式版	最佳設定	訓練預算加倍,1 個 seed	2

為什麼每組要 3 個 seed? seed 決定了隨機數,換 seed 就是換一次「運氣」。只跑 1 次,好結果可能只是運氣好;跑 3 次才看得出「穩不穩」。(後面會看到,運氣的影響其實比超參數還大。)

控制變因(公平比較的基礎):

- **訓練預算**:握拳 100 萬步、捏合 50 萬步(依先前 PPO 試驗決定),同一任務內所有設定都一樣。
- 分數、成功條件、回合長度、初始雜訊、評估方式完全相同。
- 全部用 CPU。

3.8 評估指標:怎麼算「好」?

指標	意思
最終成功率	訓練結束時的模型,用確定性策略跑 20 回合,成功幾次
平均成功步數	成功的回合平均花幾步——越少代表動作越俐落
首次 ≥ 90% 的步數	訓練過程中定期評估,第一次達到 90% 成功率時,訓練了幾步——代表**學得多快**

「最佳」的判斷規則在開跑前就寫進程式(`run_pipeline.py` 的 `select_best()`),避免看到結果之後才挑對某方有利的標準:

1. 先比最終成功率;
2. 跟第一名差 5 個百分點以內的,比「首次 ≥ 90% 的步數」,越少越好;
3. 還平手,比平均成功步數。

> **本節重點**
>
> - **坑 1**:成功條件在物理上做不到 → 訓練前先用 CEM 檢查可行性。
> - **坑 2**:數字通過但行為不對(碰到側面)→ 一定要畫出來看。
> - **坑 3**:靜態分析的排名跟實際訓練相反 → 獎勵設計要用實際訓練驗證。
> - 握拳標準姿勢**由合法樣本統計出來**:四指嚴格、拇指寬鬆。
> - GPU 在這個任務上反而比較慢。
> - 判斷規則事先固定,每組 3 個 seed。

4. 實驗結果

> **本節要回答**:哪個演算法最好?哪組超參數最好?最終模型表現如何?

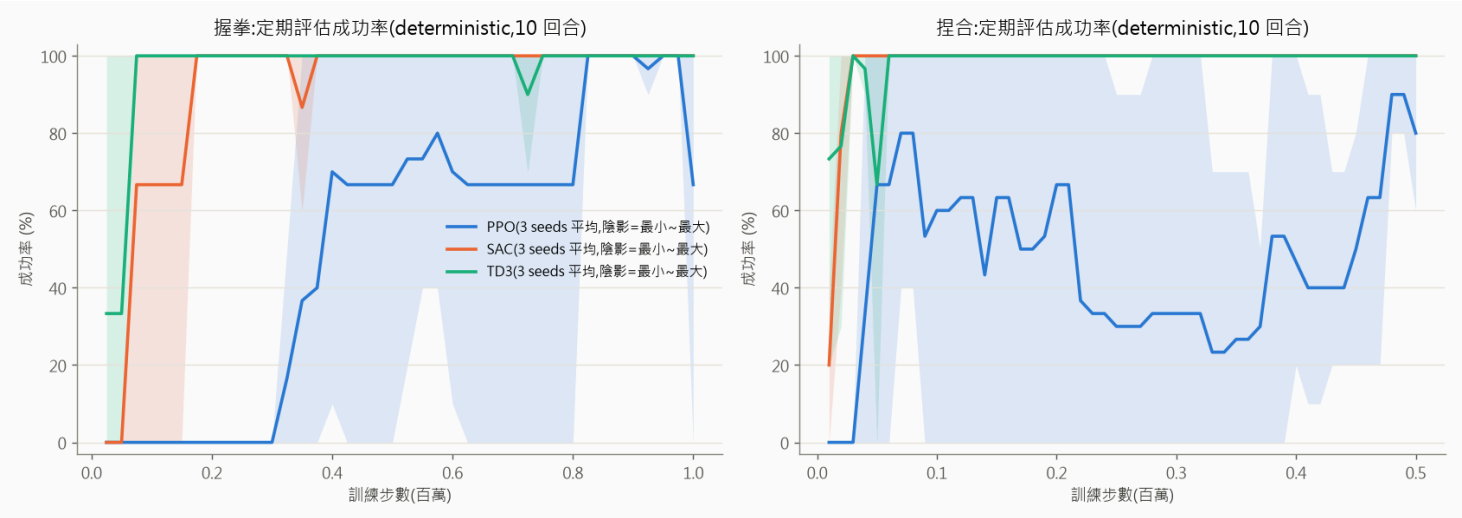
全自動流程從 2026-10-03 04:42 跑到 10-04 07:58,**共 27.25 小時、56 組訓練**,中間失敗 1 次(自動重試成功)。

> **程式對照**:整個流程由 `run_pipeline.py` 自動完成;每一組實驗由 `train_experiment.py` → `rl_framework.run_experiment()` 執行;圖表與彙整表由 `analysis/plot_results.py` 產生,表格全文在 `analysis/tables.md`。

4.1 階段 A:哪個演算法最好?

任務	演算法	最終成功率(3 個 seed)	平均成功步數	首次 ≥90% 的步數(平均)	每組花的時間
握拳	PPO	100 / 90 / 100%	約 33 步	52.5 萬步	7 分鐘
握拳	SAC	100 / 100 / 100%	約 24 步	10.8 萬步	178 分鐘
握拳	**TD3** ✅	100 / 100 / 100%	約 26 步	**5.8 萬步**	147 分鐘

任務	演算法	最終成功率(3 個 seed)	平均成功步數	首次 ≥90% 的步數(平均)	每組花的時間
捏合	PPO	100 / 75 / 70%	約 111 步	7 萬步	4 分鐘
捏合	SAC	100 / 100 / 100%	約 13 步	2.3 萬步	84 分鐘
捏合	**TD3** ✅	100 / 100 / 100%	約 13 步	**1.7 萬步**	65 分鐘



怎麼讀這張圖:橫軸是訓練了幾步,縱軸是定期評估的成功率。實線是 3 個 seed 的平均,陰影是 3 個 seed 的範圍(越寬代表運氣影響越大)。

看到了什麼:

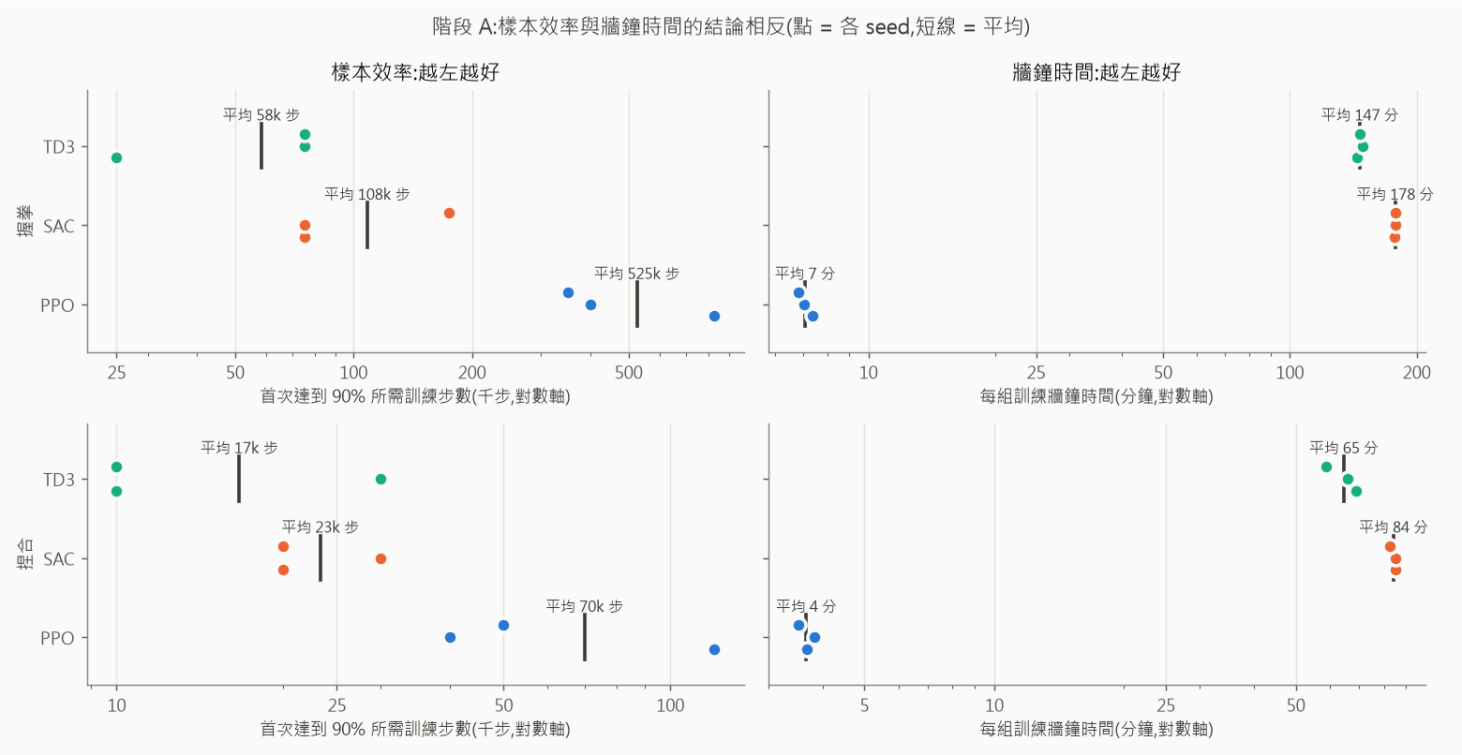
- 綠線(TD3)和橘線(SAC)在**幾萬步內**就衝到 100% 並維持住。
- 藍線(PPO)要**幾十萬步**才上來,而且在捏合任務上一直上下亂跳。

依規則,兩個任務都選出 TD3。

4.2 一個反轉:換個方式衡量,結論就變了

上面的「首次 ≥ 90% 的步數」衡量的是**看了多少經驗**。但如果問「**實際要等多久**」呢?

任務	演算法	首次 ≥90% 的步數	首次 ≥90% 實際花的時間
握拳	PPO	52.5 萬步	**3.8 分鐘** ← 最快!
握拳	SAC	10.8 萬步	19.1 分鐘
握拳	TD3	**5.8 萬步**	8.1 分鐘



左邊(步數)TD3 最好,右邊(時間)PPO 最快——兩種衡量方式幾乎完全相反!

回想第 2.3 節的表:PPO 資料只用一次,所以需要很多步,但**每一步很快**(約每秒 2,000 多步);TD3 有錯題本,需要的步數少,但**每一步都要抽題目複習**,很慢(約每秒 100 多步)。

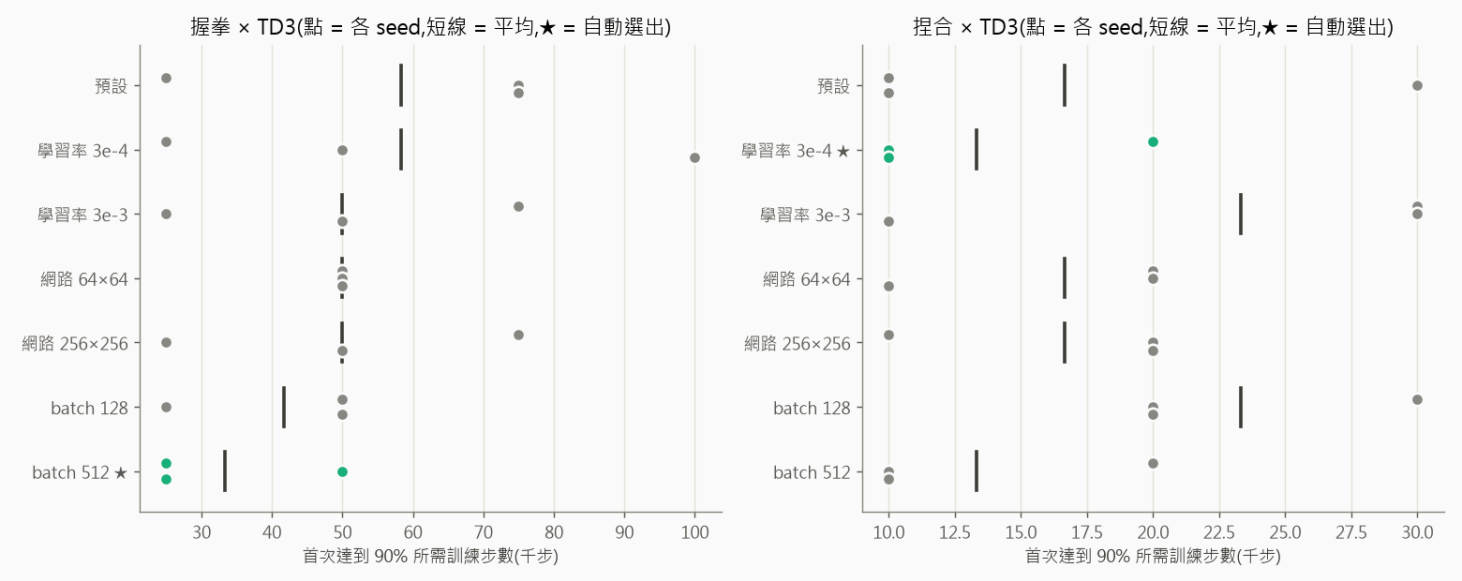
還有一個驚人的數字:TD3 握拳大約 **8 分鐘就學會了**,但因為預算是 100 萬步,它一直訓練到第 **147 分鐘**——**超過 90% 的時間都花在已經學會之後**。

4.3 階段 B:TD3 的超參數怎麼調?

全部 42 組(包含預設)的最終成功率**都是 100%**,所以比較的是「學得多快」:

設定	握拳:首次 ≥90%(3 個 seed,千步)	平均	捏合:首次 ≥90%	平均
預設	25 / 75 / 75	58,333	10 / 30 / 10	16,667
學習率 3×10^{-4}	25 / 50 / 100	58,333	20 / 10 / 10	**13,333** ✅
學習率 3×10^{-3}	75 / 25 / 50	50,000	30 / 30 / 10	23,333
網路 64×64	50 / 50 / 50	50,000	20 / 20 / 10	16,667
網路 256×256	75 / 25 / 50	50,000	10 / 20 / 20	16,667
batch 128	50 / 25 / 50	41,667	30 / 20 / 20	23,333
batch 512	25 / 50 / 25	**33,333** ✅	20 / 10 / 10	13,333

- **握拳 → batch 512**
- **捏合 → 學習率 3×10^{-4}**(跟 batch 512 平手,依第三條規則「平均成功步數」13.47 vs 13.53 步勝出——**只差 0.06 步**)



這張圖要特別仔細看:每一列是一種設定,每個點是一個 seed,短線是平均。注意**同一列的點分得多開**(例如握拳「學習率 3e-4」從 25 到 100),**而不同列的短線位置其實差不多**。也就是說:**運氣(seed)的影響比超參數還大**。

4.4 正式版模型

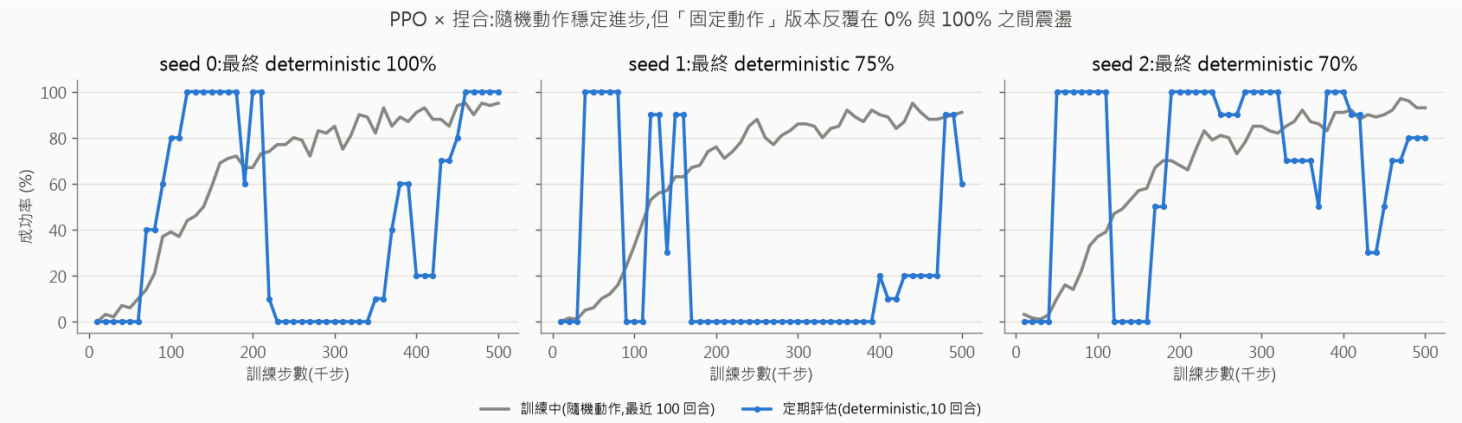
任務	設定	訓練步數	成功率	平均幾步完成
握拳	TD3 + batch 512	200 萬	**100%**(20/20)	24.1 步(約 0.24 秒)
捏合	TD3 + 學習率 3×10^{-4}	100 萬	**100%**(20/20)	13.1 步(約 0.13 秒)

想親眼看看:

```
python watch_experiment.py results/final/fist/td3/final_seed0
python watch_experiment.py results/final/pinch/td3/final_seed0
```

4.5 PPO 捏合的怪現象

PPO 捏合的最終成功率只有 75%、70%(seed 1、2),比 SAC、TD3 差很多。仔細看它的學習曲線:



- **灰線**(訓練中,帶隨機性的策略):從 0% **平穩地**爬到約 90%。
- **藍線**(評估,確定性策略):在 0% 和 100% 之間**瘋狂跳動**。例如 seed 1:4–8 萬步是 100%,17–39 萬步卻全部是 0%。

所以問題不是「學會又忘了」,而是 **PPO 的「只取平均動作」版本本身很不穩定**。

> **本節重點**
>
> - 兩個任務都是 **TD3** 勝出(看步數)。
> - **但看實際時間,握拳是 PPO 最快**——衡量方式決定結論。

> - TD3 的超參數:握拳 batch 512、捏合學習率 3×10^{-4},但**差異小於運氣的影響**。
> - 正式版模型兩個任務都 100% 成功。
> - PPO 捏合的確定性策略極不穩定,隨機策略卻穩定進步。

5. 結果討論:為什麼會這樣?

> **本節要回答**:這些結果背後的原因是什麼?有什麼意外?這次實驗的限制在哪裡?

5.1 為什麼 off-policy 的樣本效率這麼高?

因為「錯題本」。PPO 每收集一批經驗,學完就丟;TD3 和 SAC 把每一步都存起來,每一步都從裡面抽 256 筆(或 512 筆)來學。同樣一步的經驗,off-policy 可能被反覆學上百次,當然需要的步數少很多。

但天下沒有白吃的午餐:**每一步都要做一次神經網路更新**,這就是為什麼它每一步這麼慢。

5.2 那到底該選哪個?

看你的瓶頸是什麼:

情境	什麼比較貴	該選
在**真實機器人**上訓練	每一步都有時間成本、馬達會磨損、可能弄壞東西	**TD3 / SAC**(步數少)
在**模擬器**裡訓練	互動幾乎免費,電腦時間才是成本	**PPO** 可能更划算(實際時間短)

這次是在模擬器裡,所以嚴格來說,如果目標是「最快拿到一個能用的模型」,PPO 握拳 3.8 分鐘就到 90% 了。但我們事先定的規則是比步數(因為這個專案的長期目標是移轉到真實機械手),所以選了 TD3。**兩個結論都對,差別在問的問題不同。**

另外,**固定步數的預算對 off-policy 很浪費**:TD3 8 分鐘就學會了,卻跑了 147 分鐘。如果加上「**提前停止**」(連續幾次評估都 100% 就停),可以省下九成以上的時間。

5.3 為什麼超參數差異這麼小?

1. **任務相對簡單**:兩個都是「把手擺到某個姿勢」,而且分數是連續的距離(越近越好),方向很清楚。對這種任務,TD3 在預設值附近都能學好。
2. **評估的解析度有限**:握拳每 2.5 萬步才評估一次,「首次 ≥90% 的步數」只能是 2.5 萬的倍數(25k、50k、75k……),所以很容易平手。捏合的勝負最後由 0.06 步決定——**實質上跟擲銅板差不多**。

所以正確的解讀是:**「依我們事先定的規則,選出 batch 512 和學習率 3×10^{-4}」**,而不是「batch 512 明顯比較好」。

如果同時考慮時間,**64×64 的小網路**其實很有吸引力:學習效率差不多,訓練時間卻不到一半(握拳 66 分鐘 vs 預設 147 分鐘)。

5.4 為什麼 PPO 捏合的確定性策略這麼不穩?

PPO 的策略是「平均動作 + 隨機抖動」。評估時只用平均動作。捏合需要兩根手指**精準對齊**——差 1 公分就碰不到。

我們的推測(還沒有驗證):如果平均動作剛好落在「差一點點碰到」的位置,確定性評估就會**整批失敗**(20 回合全 0%);但訓練時的隨機抖動,反而有機會讓手指碰上。隨著訓練進行,平均動作在「剛好碰到」和「差一點」之間移動,於是評估結果在 0% 和 100% 之間跳。

實務意義:如果要把 PPO 學出來的捏合策略拿去用(通常是用確定性版本),它並不可靠。

5.5 為什麼沒有探索雜訊的 TD3 還學得這麼好?

照理說,確定性策略又沒有探索雜訊,應該很難發現新動作。我們的推測(未驗證):

1. 分數是連續的距離,Critic 的梯度本身就會指出「往哪個方向動會更好」,不太需要亂試。
2. 每回合的初始姿勢有隨機變化,提供了一些多樣性。

這也提醒我們:**這次的 TD3 結果不能直接代表「原始論文版的 TD3」**。

5.6 這次實驗的限制

做研究要誠實說明「哪些結論不能說」:

1. **seed 太少**:每組只有 3 個,沒有做統計檢定;正式版只有 1 個 seed。
2. **一次只改一個超參數**:沒看「學習率 + batch size 一起調」的交互作用。
3. **評估間隔造成平手**:學習速度的指標解析度不夠。
4. **判斷規則沒考慮穩定度**:「首次 ≥90%」只看第一次達標,不管之後有沒有掉下來。我們事後檢查(新增「穩定度」指標)發現 SAC、TD3 全部都很穩(達標後 96–100% 的評估仍維持 ≥90%),所以這次的結論不受影響;但如果是 PPO 這種會亂跳的,就可能誤判。
5. **PPO、SAC 沒調參數**,TD3 沒加探索雜訊——調過之後排名可能不同。
6. **PPO 用 8 個環境、SAC/TD3 用 1 個**:以步數算是公平的,但實際時間的比較會受影響。
7. **只在模擬中**:沒有在真的 ORCA Hand 上試過。
8. **任務比較簡單**:沒有抓東西、轉東西這類跟物體互動的任務。
9. **初始雜訊很小**(±3 度):沒在雜訊下訓練過的模型也能全部成功,對「穩健度」的檢驗有限。

5.7 其他工程上的小發現

- **可重現性只在同一種裝置上成立**:同一個 seed,在 CPU 上重跑兩次,結果一模一樣;但 CPU 和 GPU 的浮點運算有微小差異,會讓訓練過程逐漸分岔。我們遇過同一個設定,CPU 跑出 0%、GPU 跑出 100% 的情況(E17)。
- **最後只剩一組在跑時沒用滿 CPU**:每組固定 4 個執行緒,最後只剩握拳正式版時,CPU 只用了 25%,多花了約 1.5 小時。
- **有一組實驗失敗了**,但錯誤訊息被重試覆蓋掉,原因不明(E19)。之後已改成保留每次的錯誤訊息。

> **本節重點**
>
> - 樣本效率 vs 時間:**看你的瓶頸**。真實機器人 → TD3/SAC;模擬 → PPO 可能更划算。
> - 超參數差異小於運氣影響;勝出者是「依規則選出」,不是「明顯較好」。
> - PPO 的確定性策略在需要精準對齊的捏合任務上不可靠。
> - 誠實列出 9 項限制。

6. 結論與後續建議

> **本節要回答**:最終選定的設定是什麼?下一步可以往哪裡走?

6.1 最終選定的設定

任務	演算法	超參數	訓練步數	成果
握拳	TD3	batch size 512(其餘 SB3 預設)	200 萬	100% 成功,平均 0.24 秒
捏合	TD3	學習率 3×10^{-4}(其餘 SB3 預設)	100 萬	100% 成功,平均 0.13 秒

模型檔在 `results/final/<任務>/td3/final_seed0/model.zip` 。

6.2 這次學到的五件事

1. **先檢查任務在物理上做不做得到**,再開始訓練。(握拳和捏合 v1 都做不到)
2. **把結果畫出來看**,數字通過不代表行為正確。(碰到側面也算捏合)
3. **獎勵設計要用實際訓練驗證**,靜態分析可能完全相反。(拇指項)
4. **「最好」取決於你怎麼衡量**——步數還是時間。(TD3 vs PPO)
5. **多跑幾個 seed**,運氣的影響可能比你想調的參數還大。(階段 B)

6.3 後續建議

短期(改進這次的實驗):

- 加入**提前停止**,用「達到門檻的實際時間」作為正式比較指標之一。
- 判斷規則加入**穩定度條件**(達標後要維持住)。
- **增加 seed 數**(例如 5–10 個),並報告信賴區間。
- 給 TD3 加上探索雜訊、給 PPO 和 SAC 調參數,再比一次。

中期(擴展任務):

- 加入**跟物體互動**的任務:抓起一顆球、在手中轉動方塊。這類任務的分數不像「距離」那麼單純,演算法之間的差異可能會更明顯。
- **加大初始雜訊**,並加入**領域隨機化**(domain randomization:隨機改變摩擦力、質量、馬達強度等),讓策略更穩健。

長期(走向真實機器人):

- 把模擬中學到的策略移到**真實的 ORCA Hand** 上(sim-to-real)。到那時,每一步的互動都有成本,off-policy 的樣本效率優勢會更有價值。
- 用訓練好的策略產生的握拳姿勢,再反推一次「標準握拳」,看看跟 CEM 找到的有什麼不同。

7. 課後思考題

用來檢查自己是否真的理解。參考答案的線索在括號裡的章節。

1. 為什麼 v1 的握拳任務,即使用最好的演算法、訓練再久,成功率也會是 0%?關閉碰撞那個實驗證明了什麼?(3.1)
2. 捏合的成功條件從「表面距離 < 2 mm」改成「指尖點距離 < 10 mm 且表面接觸」,是為了排除哪一種情況?如果只看數字,這個問題為什麼發現不了?(3.2)
3. 握拳標準姿勢的「容許範圍」為什麼拇指比四指大很多?這跟 v1 失敗的原因有什麼關係?(3.3)
4. 拇指距離項在靜態分析中 AUC 最高,實際訓練卻最差。用「學習過程的先後順序」解釋為什麼。(3.4)
5. 如果你要在**真實的機械手**上訓練(每一步都要真的動馬達),你會選 PPO 還是 TD3?如果是在模擬器裡、只想最快拿到能用的模型呢?(5.2)
6. 階段 B 中,捏合的最佳設定是由 0.06 步的差距決定的。你會怎麼向指導教授說明這個結果的可信度?(5.3)
7. PPO 捏合的訓練成功率穩定上升,評估成功率卻一直跳。如果要把這個模型實際拿來用,你會用確定性還是隨機性的動作?為什麼?(5.4)
8. 為什麼 GPU 在這個專案裡反而比 CPU 慢?在什麼樣的情況下 GPU 才會有幫助?(3.6)
9. 「首次 ≥ 90% 的步數」這個指標有什麼漏洞?我們用什麼方法確認它在這次實驗中仍然可靠?(5.6 第 4 點)
10. 如果只能再多做一個實驗來加強這次的結論,你會做什麼?為什麼?(5.6、6.3)

8. 附錄:常用指令與程式地圖

8.1 常用指令(在專案資料夾、 `conda activate orca` 之後執行)

想做什麼	指令
看最終的握拳 / 捏合模型	`python watch_experiment.py results/final/fist/td3/final_seed0`
看 v1 和 v2 任務定義的差別(開視窗)	`python demo_reward_v2.py --view`
跑一組實驗	`python train_experiment.py --task pinch --algo td3 --seed 0 --hp learning_rate=3e-4`
用 GPU 跑	加上 `--device cuda`
只列出完整實驗的計畫和估計時間	`python run_pipeline.py --dry-run`
重新產生所有圖表與表格	`python analysis/plot_results.py`
重現「v1 做不到」的檢查	`python analysis/verify_v1_infeasible.py`
重現獎勵一致性分析	`python analysis/reward_consistency.py`

8.2 程式地圖

檔案	它在做什麼	重要的函式 / 類別
`fist_task_v2.py`	握拳任務:怎麼給分、 怎麼判定成功	`OrcaFistTaskV2` 、 `pose_error()` 、 `tip_distances()` 、 `_is_fist_pose()`
`pinch_task_v2.py`	捏合任務	`OrcaPinchTaskV2` 、 `fingertip_distance()` 、 `contact_distance()` 、 `_is_pinch()`
`task_utils.py`	兩個任務共用的量測工具	`surface_distance()` 、 `fingertip_local_point()` 、 `apply_reset_noise()`
`rl_framework.py`	統一的訓練 + 評估框架	`ExperimentConfig` (所有設定)、 `run_experiment()` (跑一組)、 `evaluate_policy()` (評估)

檔案	它在做什麼	重要的函式 / 類別
`train_experiment.py`	從命令列跑一組實驗	`--task` 、 `--algo` 、 `--hp` 、 `--device`
`run_pipeline.py`	全自動跑完所有實驗並自動選最佳	`run_jobs()` (平行排程)、 `select_best()` (判斷規則)
`analysis/plot_results.py`	產生圖表和表格	`stability()` (穩定度)、 `minutes_to_first90()` (達標時間)
`derive_fist_standard.py`	從合法樣本反推握拳標準姿勢	`collect_samples()`
`demo_reward_v2.py`	v1/v2 比較圖、CEM 工具	`cem()` 、 `settle()` 、 `run_until_done()`

8.3 延伸閱讀

- `實驗紀錄.md` :每個實驗的完整數據、用到的函式、失敗的嘗試(E01–E23),開頭有完整的名詞解釋。
- `論文_ORCA機械手強化學習演算法與超參數比較.md` :同一批結果的正式學術寫法,含公式與參考文獻。
- `analysis/tables.md` :所有實驗的完整數字表。
- Sutton & Barto,《Reinforcement Learning: An Introduction》(第 2 版):強化學習的經典入門教科書,第 1 章與第 3 章對應本講義第 2.1 節。